计算机学院_社交媒体与舆情分析_课程实验报告

实验题目：朴素贝叶斯算法		学号：202200130077
日期：2025.9.17	班级： 人工智能	姓名： 薛润泽
Email：		

实验方法介绍：
本次实验的核心任务是利用朴素贝叶斯算法对电影评论进行情感倾向（正面/负面）的二元分类。
朴素贝叶斯是一种基于贝叶斯定理的简单概率分类算法。其核心思想是，对于给定的待分类项，求解在此项出现的情况下，它属于哪个类别的概率最大。对于文本分类问题，待分类项是一篇文档（评论），类别是情感倾向（正面/负面）。
贝叶斯定理如下：

$$P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)}$$

朴素贝叶斯算法做了一个关键的“特征条件独立性假设”。在文本分类中，这意味着假设文档中的每个词的出现都独立于其他词。

$$P(d|c) = P(w_1, w_2, ..., w_n|c) = \prod_{i=1}^{n} P(w_i|c)$$

最终，分类公式为：

$$\hat{c} = \arg\max_{c} P(c) \cdot \prod_{i=1}^{n} P(w_i|c)$$

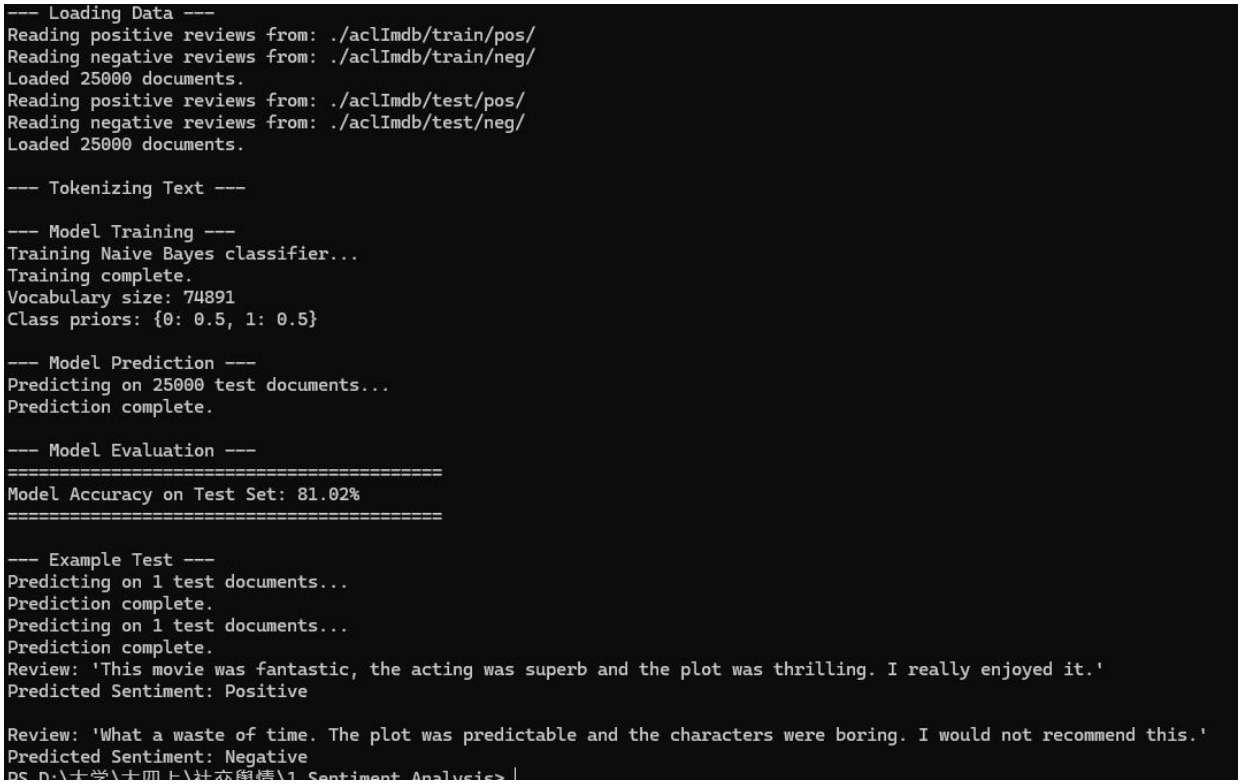
实验过程描述：（不要求罗列完整源代码）

1、 数据加载与预处理

（1）文件读取：通过 load_data 函数分别读取训练集和测试集的 pos 和 neg 目录下的所有.txt 文件。
（2）文本转换：将所有评论文本内容转换为小写，以减少词汇量并统一表示。
（3）标签分配：为 pos 目录下的评论分配标签 1，为 neg 目录下的评论分配标签 0。
（4）数据混洗：将数据和标签合并后进行随机混洗以消除数据顺序对训练可能造成的影响。
2、使用 tokenize 函数对每条评论进行分词，将文本转化为词语列表

3、模型训练：
（1）将朴素贝叶斯分类器初始化后，遍历整个训练集，统计每个类别文本数量除以总文档数，得到鲜艳概率
（2）统计次品，遍历训练集中所有 文档以及对应标签
4、模型预测，遍历测试样本，对测试集中每条已分词的评论进行预测。计算后验概率。
5、模型评估：计算正确率为模型性能评估指标。

结论分析：
输出结果如下：



可以看到，模型对包含大量正面词汇和负面词汇的不同评论分类逻辑正确。其预测能力可靠。

结论：
模型实现成功，分类正确率达到 81.02%。
代码附录：

```
import os
import re
import numpy as np
from collections import defaultdict
from random import shuffle
import math


def load_data(data_dir):
    pos_path = os.path.join(data_dir, 'pos/')
```

```
    neg_path = os.path.join(data_dir, 'neg/')

    pos_files = [pos_path + f for f in os.listdir(pos_path) if f.endswith('.txt')]
    neg_files = [neg_path + f for f in os.listdir(neg_path) if f.endswith('.txt')]

    print(f"Reading positive reviews from: {pos_path}")
    pos_list = [open(f, 'r', encoding='utf-8').read().lower() for f in pos_files]
    print(f"Reading negative reviews from: {neg_path}")
    neg_list = [open(f, 'r', encoding='utf-8').read().lower() for f in neg_files]

    data = pos_list + neg_list
    labels = [1] * len(pos_list) + [0] * len(neg_list)

    merged = list(zip(data, labels))
    shuffle(merged)
    data, labels = zip(*merged)

    print(f"Loaded {len(data)} documents.")
    return list(data), list(labels)

def tokenize(text):

    return re.findall(r'\b\w+\b', text)

class NaiveBayesClassifier:

    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.class_priors = {}
        self.word_counts = {}
        self.class_total_words = {}
        self.vocab = set()

    def fit(self, X_train, y_train):
        print("Training Naive Bayes classifier...")
        num_docs = len(X_train)
        self.classes = np.unique(y_train)
        for c in self.classes:
            self.word_counts[c] = defaultdict(int)
            self.class_total_words[c] = 0
        for x, y in zip(X_train, y_train):
            for word in x:
                self.word_counts[y][word] += 1
                self.class_total_words[y] += 1
```

```
                self.vocab.add(word)
        for c in self.classes:
            num_class_docs = sum(1 for label in y_train if label == c)
            self.class_priors[c] = num_class_docs / num_docs

        print("Training complete.")
        print(f"Vocabulary size: {len(self.vocab)}")
        print(f"Class priors: {self.class_priors}")

    def predict(self, X_test):
        print(f"Predicting on {len(X_test)} test documents...")
        predictions = []
        for x in X_test:
            scores = {c: math.log(self.class_priors[c]) for c in self.classes}
            for word in x:
                if word in self.vocab:
                    for c in self.classes:
                        numerator = self.word_counts[c].get(word, 0) + self.alpha
                        denominator = self.class_total_words[c] + self.alpha *
len(self.vocab)
                        scores[c] += math.log(numerator / denominator)
            predicted_class = max(scores, key=scores.get)
            predictions.append(predicted_class)

        print("Prediction complete.")
        return predictions

if __name__ == "__main__":
    base_dir = './aclImdb/'
    train_dir = os.path.join(base_dir, 'train/')
    test_dir = os.path.join(base_dir, 'test/')
    print("--- Loading Data ---")
    X_train_raw, y_train = load_data(train_dir)
    X_test_raw, y_test = load_data(test_dir)
    print("\n--- Tokenizing Text ---")
    X_train_tokens = [tokenize(text) for text in X_train_raw]
    X_test_tokens = [tokenize(text) for text in X_test_raw]
    print("\n--- Model Training ---")
    nb_classifier = NaiveBayesClassifier(alpha=1.0)
    nb_classifier.fit(X_train_tokens, y_train)
    print("\n--- Model Prediction ---")
    predictions = nb_classifier.predict(X_test_tokens)
    print("\n--- Model Evaluation ---")
    predictions = np.array(predictions)
```

```
    y_test = np.array(y_test)
    accuracy = np.mean(predictions == y_test)
    print(f"========================================")
    print(f"Model Accuracy on Test Set: {accuracy * 100:.2f}%")
    print(f"========================================")
    print("\n--- Example Test ---")
    test_review_pos = "This movie was fantastic, the acting was superb and the plot
was thrilling. I really enjoyed it."
    test_review_neg = "What a waste of time. The plot was predictable and the
characters were boring. I would not recommend this."
    pos_tokens = tokenize(test_review_pos.lower())
    neg_tokens = tokenize(test_review_neg.lower())
    pos_prediction = nb_classifier.predict([pos_tokens])[0]
    neg_prediction = nb_classifier.predict([neg_tokens])[0]
    print(f"Review: '{test_review_pos}'")
    print(f"Predicted Sentiment: {'Positive' if pos_prediction == 1 else
'Negative'}\n")
    print(f"Review: '{test_review_neg}'")
    print(f"Predicted Sentiment: {'Positive' if neg_prediction == 1 else
'Negative'}")
```